Dauphine UNIVERSITÉ PARIS SCIENCES & LETTRES

PROJET DE NATURAL LANGUAGE PROCESSING

ParaDetox

Génération de paraphrases non toxiques

Étude comparative d'architectures Encoder–Decoder implémentées et entraînées depuis zéro avec PyTorch

Rapport technique

Master 2 Intelligence Artificielle & Data Science

Présenté par **ROMDHANI Soumaya**

Formation Master 2 Intelligence Artificielle & Data Science

Université Université Paris Dauphine–PSL

Année universitaire 2026–2027 Octobre 2026

Résumé

La détoxification de texte consiste à transformer un énoncé offensant en une reformulation moins toxique tout en préservant son contenu. Ce rapport étudie cette tâche comme un problème de génération *sequence-to-sequence* et compare trois architectures construites et entraînées depuis zéro : un *BiLSTM Encoder–Decoder* de référence, sa variante avec attention et une variante avec mécanisme de copie *Pointer*. Les expériences utilisent ParaDetox [1] : 19 003 paires nettoyées, séparées en apprentissage, validation et test sans recouvrement textuel exact selon la procédure de groupement implémentée.

Les modèles sont évalués à l'aide de BLEU, chrF, de la toxicité prédite, de la similarité sémantique et de diagnostics de préservation de l'information. Sur 1 193 sources de test, le système *Pointer + Safe Decoding*, retenu selon la validation, atteint **66,59 BLEU**, **79,88 chrF** et **66,81 %** selon un indicateur conjoint automatique. Ce dernier n'est *pas* un taux de réussite évalué par des annotateurs humains. L'analyse qualitative révèle encore des erreurs de grammaire, d'expressions idiomatiques, de fidélité et de détoxification. Le projet montre à la fois l'intérêt des mécanismes d'attention et de copie, et les limites des générateurs récurrents de petite taille.

Mots-clés : génération de texte, détoxification, paraphrase, LSTM, attention, pointer-generator, apprentissage supervisé, évaluation NLP.

Table des matières

1 Introduction **3**

1.1 Contexte et motivation . . . 3

1.2 Formulation du problème . . . 3

1.3 Objectifs et contributions du projet . . . 3

2 Concepts et travaux liés **3**

2.1 Génération Encoder–Decoder . . . 3

2.2 Attention et mécanisme Pointer . . . 3

2.3 Détoxification et apprentissage guidé . . . 3

3 Jeu de données : ParaDetox **4**

3.1 Provenance, champs et taille . . . 4

3.2 Évitement des fuites entre partitions . . . 4

3.3 Augmentation limitée à l'entraînement . . . 4

4 Prétraitement et représentation des tokens **4**

4.1 Nettoyage et normalisation . . . 4

4.2 Tokenisation et vocabulaire . . . 5

4.3 Représentation vectorielle . . . 5

5 Architectures proposées **5**

5.1 Vue d'ensemble . . . 5

5.2 Modèle A : baseline BiLSTM Encoder–Decoder . . . 5

5.3 Modèle B : BiLSTM avec attention 6
5.4 Modèle C : attention et mécanisme Pointer 6

6 Protocole d'entraînement 6
6.1 Fonction objectif 6
6.2 Hyperparamètres reproductibles 6
6.3 Extension et scheduled sampling 7
6.4 Courbes de convergence 7

7 Résultats expérimentaux 8
7.1 Protocole de sélection et inférence 8
7.2 Définition des mesures 8
7.3 Comparaison des modèles sur le test 8
7.4 Diagnostics complémentaires et interprétation 9

8 Exemples générés et analyse qualitative 9
8.1 Réussites représentatives observées 9
8.2 Échecs représentatifs et causes 9
8.3 Analyse des mécanismes d'erreur 10

9 Discussion critique 10
9.1 Réponses aux questions expérimentales 10
9.2 Limites méthodologiques 11
9.3 Améliorations futures 11

10 Conclusion 11

Traçabilité et reproductibilité 11

1. Introduction

1.1 Contexte et motivation

La reformulation automatique de contenus offensants est pertinente pour les assistants conversationnels, la modération assistée et les outils d'aide à la rédaction. À la différence d'une simple classification *toxique / non toxique*, une tâche de détoxification doit *produire une nouvelle phrase*. Il est insuffisant de retirer l'insulte si cette suppression détruit la syntaxe ou modifie une date, un montant ou une négation.

1.2 Formulation du problème

Soit $x = (x_1, \ldots, x_n)$ une phrase potentiellement toxique et $y = (y_1, \ldots, y_m)$ sa reformulation. Un modèle paramétré par θ approxime la distribution conditionnelle :

$$p_\theta(y \mid x) = \prod_{t=1}^{m} p_\theta(y_t \mid y_{<t}, x). \tag{1}$$

Le résultat recherché doit satisfaire trois objectifs liés mais non équivalents : **diminuer la toxicité**, **conserver le sens et les faits** et **rester grammaticalement correct**.

Exemple de tâche. Entrée : *"Your idea is stupid."*
Sortie effectivement observée : *"Your idea is bad."*
L'opinion reste négative, mais l'attaque directe est atténuée.

1.3 Objectifs et contributions du projet

Ce travail vise à (i) construire toute la chaîne de génération sans fine-tuning d'un générateur préentraîné ; (ii) comparer une baseline et des améliorations architecturales ; (iii) étudier le décodage greedy, beam et le reclassement de candidats ; (iv) mesurer les performances quantitatives et interpréter les succès comme les échecs.

2. Concepts et travaux liés

2.1 Génération Encoder–Decoder

Le paradigme Encoder–Decoder transforme une séquence source en représentations latentes qu'un décodeur exploite pour prédire chaque token de sortie [2]. Les réseaux *Long Short-Term Memory* (LSTM) permettent de conserver une partie du contexte au moyen de portes récurrentes, mais la compression de toute la source dans un unique vecteur peut devenir limitante.

2.2 Attention et mécanisme Pointer

L'attention résout partiellement cette difficulté en permettant au décodeur d'accéder aux états successifs de l'encodeur [3]. Un mécanisme Pointer combine une distribution de génération et une distribution de copie à partir de la source [4]. Il peut ainsi faciliter la conservation de sous-mots, mais risque également de recopier du contenu offensant.

2.3 Détoxification et apprentissage guidé

Le corpus ParaDetox fournit des paires supervisées toxique–neutre [1]. Le *teacher forcing* accélère l'optimisation en alimentant le décodeur avec les tokens de référence ; à l'inférence, les tokens précédents proviennent au contraire du modèle. Ce décalage, appelé *exposure bias*, motive l'expérience exploratoire de *scheduled sampling* [5].

Restriction du sujet respectée : les modèles qui *génèrent* les phrases ont été initialisés aléatoirement et entraînés sur ParaDetox. Des modèles préentraînés sont employés *uniquement comme évaluateurs* de toxicité et de similarité, sans fournir de génération textuelle.

3. Jeu de données : ParaDetox

3.1 Provenance, champs et taille

Les données proviennent du dépôt public `s-nlp/paradetox`, révision `958b799`. Le fichier contient deux champs : `en_toxic_comment` (x) et `en_neutral_comment` (y). Les paires peuvent proposer plusieurs références neutres pour une même source. Le notebook vérifie l'intégrité du téléchargement par une empreinte SHA-256.

Table 1 – Statistiques de l'expérience exécutée. Les sources distinctes sont comptées après normalisation.

Partition	Paires	Sources distinctes
Entraînement (initial)	15 201	9 541
Validation	1 901	1 193
Test	1 901	1 193
Total après nettoyage	**19 003**	—

Le fichier brut comporte **19 744** paires. Après normalisation, élimination des entrées vides et déduplication, **19 003** paires restent disponibles, soit 741 de moins. Le découpage est d'environ 80/10/10, avec une graine fixée à 42.

3.2 Évitement des fuites entre partitions

Le code relie dans un même groupe les paires dont les textes source ou cible deviennent identiques après normalisation et retrait des signes non alphanumériques. Les composantes connexes sont ensuite réparties dans les trois partitions ; une assertion vérifie l'absence de recouvrement exact suivant cette clé. Ce protocole limite les fuites par doublons, mais **ne garantit pas** l'absence de paraphrases très proches entre partitions.

3.3 Augmentation limitée à l'entraînement

Un apprentissage complémentaire sur les entrées déjà neutres vise à limiter les réécritures inutiles. La partition d'entraînement inclut 1 216 paires supplémentaires de type *neutre* → *neutre* et 406 exemples rééchantillonnés contenant notamment chiffres ou négations. La taille effective devient **16 823 exemples** (les 406 rééchantillonnages ne sont pas 406 nouveaux exemples uniques). **Validation et test restent inchangés.**

4. Prétraitement et représentation des tokens

4.1 Nettoyage et normalisation

Les phrases passent par une normalisation Unicode NFKC, la fusion des espaces multiples et la conversion en minuscules. Les signes de ponctuation et les négations sont conservés. La casse des noms propres est donc perdue pour l'apprentissage. Le traitement `readable()` utilisé lors des mesures reformate de manière *symétrique* prédictions et références ; il ne corrige pas le générateur.

4.2 Tokenisation et vocabulaire

Le tokenizer *SentencePiece BPE* [6] est entraîné exclusivement sur les textes de l'entraînement initial, avec une taille demandée de **2 000 sous-mots**. Cette représentation cherche un compromis entre robustesse aux mots rares et taille de la couche de sortie. Quatre identifiants spéciaux sont réservés :

PAD	UNK	BOS	EOS
0	1	2	3

Chaque source est encodée avec un token final `EOS`. Chaque cible commence par `BOS` et se termine par `EOS`. Les séquences d'un batch sont complétées par `PAD`, exclu de la perte ; les tokens inconnus sont représentés par `UNK`.

4.3 Représentation vectorielle

Les identifiants deviennent des embeddings entraînables de dimension $d_e = 96$. Le notebook n'impose pas de troncature fixe aux phrases source pendant l'entraînement ; à l'inférence, la génération est arrêtée à `EOS` ou après au plus **64 pas de génération**. Ce plafond peut provoquer certaines sorties incomplètes.

5. Architectures proposées

5.1 Vue d'ensemble

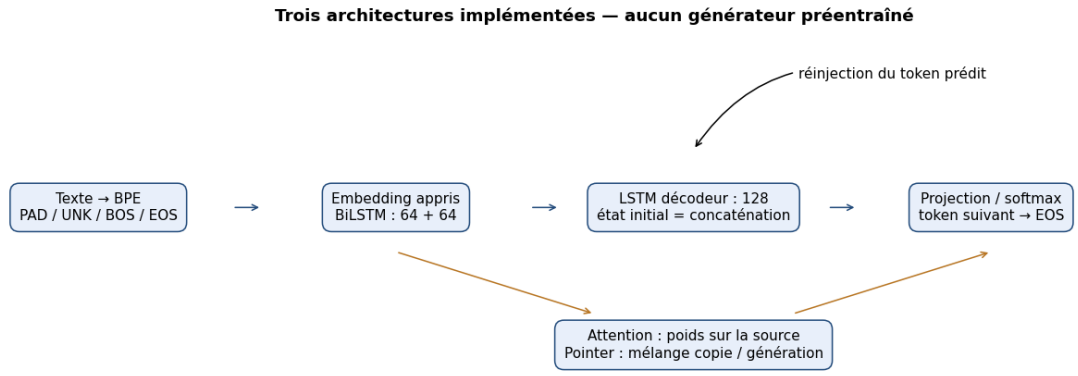


Figure 1 – Chaîne Encoder–Decoder et variantes représentées dans le notebook exécuté.

5.2 Modèle A : baseline BiLSTM Encoder–Decoder

L'encodeur est un **BiLSTM à une couche**, avec 96 unités cachées par direction et une représentation concaténée de taille 192. Les états finaux caché et cellule initialisent le décodeur LSTM à une couche ($d_h = 192$). Pour un token précédent y_{t-1}, le décodeur prédit :

$$(s_t, c_t) = \mathrm{LSTM}(E(y_{t-1}), (s_{t-1}, c_{t-1})), \qquad p_t = \mathrm{softmax}(W s_t + b). \tag{2}$$

Le modèle de référence ne dispose pas d'un accès dynamique à tous les états de la source ; il dépend principalement de l'état final de l'encodeur.

5.3 Modèle B : BiLSTM avec attention

Une transformation linéaire de l'état du décodeur produit une requête $q(s_t)$. Pour chaque position i de la source :

$$e_{t,i} = \frac{q(s_t)^\top h_i}{\sqrt{d_h}}, \qquad \alpha_{t,i} = \mathrm{softmax}_i(e_{t,i}), \tag{3}$$

$$z_t = \sum_i \alpha_{t,i} h_i, \qquad p_t = \mathrm{softmax}(W[s_t; z_t] + b). \tag{4}$$

Les positions de padding sont masquées avant la normalisation. Le contexte z_t dépend ainsi du token généré.

5.4 Modèle C : attention et mécanisme Pointer

Le mécanisme de copie apprend une porte $g_t \in [0, 1]$ et combine deux distributions :

$$p(w \mid t) = g_t \, p_{\text{vocab}}(w \mid t) + (1 - g_t) \sum_{i : x_i = w} \alpha_{t,i}. \tag{5}$$

La somme sur les positions source est implémentée avec `scatter_add` : plusieurs occurrences du même sous-mot contribuent à sa probabilité. Ce modèle vise une meilleure conservation des informations, mais une copie élevée peut laisser subsister une insulte.

Table 2 – Complexité réelle et checkpoint retenu (graine 42).

Modèle	Paramètres entraînables	Époque retenue
Baseline BiLSTM	949 712	14
BiLSTM + Attention	1 370 576	14
BiLSTM + Attention + Pointer	1 371 057	7
Attention prolongée (même architecture)	1 370 576	25

L'expérience *Attention prolongée* partage l'architecture du modèle B et modifie le budget d'entraînement. Elle n'est donc pas comptée comme une quatrième architecture indépendante.

6. Protocole d'entraînement

6.1 Fonction objectif

Les modèles minimisent la somme des log-vraisemblances négatives des tokens cible, sans compter `PAD` :

$$\mathcal{L}(\theta) = -\frac{1}{N_{\text{non-PAD}}} \sum_{b,t \,:\, y_{b,t} \neq \text{PAD}} \log p_\theta(y_{b,t} \mid y_{b,<t}, x_b). \tag{6}$$

La fonction `NLLLoss` est utilisée sur les log-probabilités fournies par le décodeur, y compris pour la distribution mixte du Pointer. Le *teacher forcing* est initialement fixé à 100 %.

6.2 Hyperparamètres reproductibles

Le scheduler `ReduceLROnPlateau` diminue le taux d'apprentissage si la loss de validation stagne (facteur 0,5, patience 1). Le meilleur checkpoint minimise la NLL de validation ; le dernier checkpoint permet la reprise dans le même environnement. Les trois architectures initiales sont comparées avec le même budget maximal.

Table 3 – Hyperparamètres du budget commun, extraits du code et des sorties de l'exécution.

Paramètre	Valeur	Paramètre	Valeur
Embeddings	96	État caché décodeur	192
Couches LSTM	1	Tokenizer BPE	2 000
Batch size	64	Taux d'apprentissage initial	0,002
Optimiseur	AdamW	Weight decay	10^{-4}
Dropout	0,25	Gradient clipping	1,0
Budget initial	14 époques	Patience (early stopping)	4
Amélioration minimale NLL	0,005	Graine	42
Génération maximale	64 tokens	Device observé	NVIDIA Tesla T4

6.3 Extension et scheduled sampling

Une expérience distincte poursuit l'entraînement du modèle Attention jusqu'à **25 époques** au maximum, avec patience 5. Le taux de teacher forcing peut décroître de 1,0 vers 0,85 après la douzième époque : le décodeur reçoit alors ponctuellement sa propre prédiction au lieu du token vrai. **La durée d'entraînement et cette stratégie évoluent ensemble ; leurs effets ne peuvent pas être isolés causalement** avec ce protocole.

6.4 Courbes de convergence

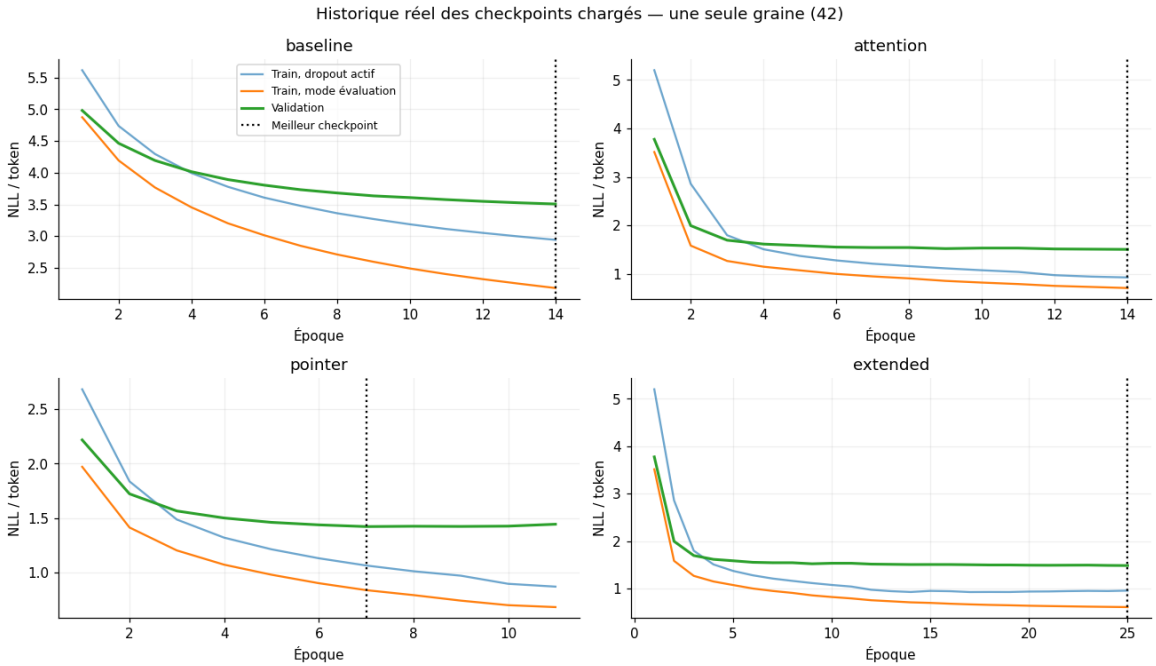


Figure 2 – NLL par token sur l'entraînement et la validation ; sorties originales du notebook. Les pointillés désignent l'époque du meilleur checkpoint chargé.

Les pertes diminuent nettement au début, surtout pour les variantes avec attention. Le modèle Pointer atteint son meilleur point de validation avant la fin du budget, ce qui justifie l'arrêt/choix du meilleur checkpoint. L'écart train–validation doit être interprété avec prudence : dropout et mode évaluation changent les pertes mesurées. La tendance n'établit pas, à elle seule, une mesure complète du surapprentissage.

7. Résultats expérimentaux

7.1 Protocole de sélection et inférence

L'inférence commence avec `BOS`, interdit `PAD` et `BOS` parmi les sorties, puis s'arrête à `EOS` ou au plafond de 64 pas. La stratégie *greedy* choisit le token le plus probable à chaque étape. Le *beam search* conserve plusieurs hypothèses, avec une pénalisation de longueur :

$$\text{score}(y) = \frac{\sum_t \log p_\theta(y_t \mid y_{<t}, x)}{((5 + |y|)/6)^{0,6}}. \tag{7}$$

Des variantes interdisent la répétition de trigrammes. Le *Safe Decoding* conserve des candidats issus de la génération neurale et applique un reclassement heuristique favorisant notamment la conservation des nombres et de certains noms, la complétude, et des combinaisons de mots plus plausibles au regard du train. **Aucune traduction de référence n'est injectée au décodage.**

Le choix du système est réalisé sur **250 sources de validation** au moyen de contraintes de qualité et du score conjoint ; il est confirmé sur les **1 193 sources complètes de validation**. Les métriques finales sont ensuite calculées sur les **1 193 sources distinctes du test**. Les jeux de diagnostic employés lors du développement restent distincts de cette estimation de population, mais les itérations successives peuvent introduire un biais exploratoire qu'il faut reconnaître.

7.2 Définition des mesures

— **BLEU et chrF** : recouvrement lexical et caractère avec les références neutres disponibles ; une paraphrase valide peut différer d'une référence.

— **Toxicité moyenne** : probabilité de classe toxique donnée par un classifieur RoBERTa public, non une annotation humaine.

— **Similarité cosinus** : proximité des embeddings `all-MiniLM-L6-v2` entre source et génération ; elle peut manquer une inversion de sens.

— **Répétitions** : proportion de phrases contenant un bigramme dupliqué.

— **Proxy conjoint** : proportion satisfaisant des conditions prédéfinies (toxicité $< 0,5$, cosinus $\geq 0,8$, sortie non vide, absence de répétition ou de troncature, contrôles numériques, de noms et de longueur). *Ce n'est pas la part de phrases grammaticalement parfaites.*

— **NLL et perplexité** : probabilité des *références* conditionnée par la source ; la perplexité est exp(NLL).

7.3 Comparaison des modèles sur le test

Table 4 – Mesures réellement enregistrées sur les 1 193 sources de test. ↓ : plus faible est préférable ; ↑ : plus élevé est préférable.

Système	BLEU↑	chrF↑	Toxicité↓	Cosinus↑	Proxy↑
Copie de l'entrée	56,86	78,69	0,990	1,000	0,08 %
Baseline greedy	8,31	26,20	0,022	0,277	0,59 %
Attention greedy	63,44	78,76	0,156	0,862	59,35 %
Pointer greedy	63,81	78,69	0,234	0,879	56,50 %
Pointer + Safe	**66,59**	**79,88**	**0,173**	**0,890**	**66,81 %**

Le modèle final retenu est `pointer_safe`. Par rapport à `attention_greedy`, il améliore le proxy conjoint d'environ **7,46 points de pourcentage**. L'intervalle bootstrap descriptif du proxy

final est approximativement [**64,20 % ; 69,41 %**], conditionnel à la graine d'entraînement et aux sources testées : il ne représente pas la variabilité entre différents entraînements.

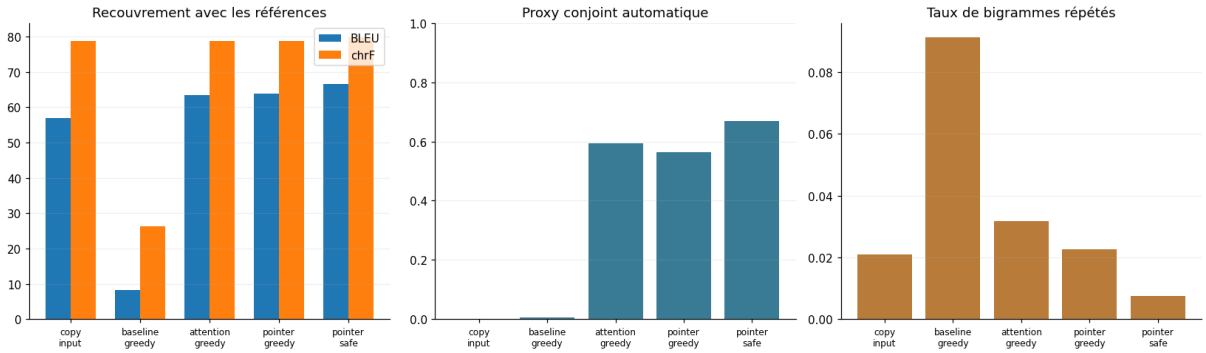


Figure 3 – Comparaisons quantitatives exportées du notebook exécuté.

7.4 Diagnostics complémentaires et interprétation

Le système final présente un taux de répétition de bigrammes d'environ **0,75 %**. Parmi les **58 sources de test contenant au moins un chiffre**, les suites numériques détectées sont conservées selon la règle automatique dans **100 %** des cas. Ce résultat *ne garantit pas* la conservation des relations entre valeurs, des dates complètes ni de la grammaire ; un diagnostic fondé sur des expressions régulières peut manquer des erreurs.

La baseline obtient une toxicité prédite faible (0,022), mais un BLEU et une similarité très faibles. Elle semble souvent réduire le problème en détruisant le contenu. À l'inverse, la copie intégrale conserve parfaitement la source mais conserve presque toute sa toxicité. Ces deux extrêmes montrent pourquoi plusieurs métriques sont indispensables.

8. Exemples générés et analyse qualitative

8.1 Réussites représentatives observées

Les exemples du tableau 5 sont des **sorties réellement enregistrées** par le modèle sélectionné dans la batterie diagnostique, et non des phrases écrites manuellement. Ils illustrent une reformulation simple réussie ; ils ne constituent pas un échantillon représentatif pour calculer un taux global de réussite.

Table 5 – Exemples qualitatifs satisfaisants (sorties observées).

Entrée	Sortie observée
Your idea is stupid.	*Your idea is bad.*
This explanation is garbage.	*This explanation is bad.*
This damn computer keeps crashing.	*This computer keeps crashing.*
What the hell are you doing ?	*What are you doing ?*
This is a shitty situation.	*This is a bad situation.*

8.2 Échecs représentatifs et causes

Le sujet universitaire demande explicitement des *failure cases*. Le tableau 6 présente donc quelques sorties brutes, sans les masquer ni les remplacer par des reformulations idéales.

Table 6 – Erreurs réellement produites par le système final dans les tests diagnostiques.

Entrée	Sortie observée	Erreur identifiée
Stop acting like an idiot.	*Stop acting like an.*	Phrase incomplète ; détoxification non grammaticale.
What a dumb decision !	*What an decision !*	Article erroné ; manque de reconstruction syntaxique.
These idiots ruined the meeting.	*These ruin ruined the meeting.*	Mot inadapté et redondance sémantique.
She worked her ass off to prepare the presentation.	*She worked her off to prepare the presentation.*	Expression idiomatique fragmentée.
The stupid report says 18.5% instead of 15.8%.	*The stupid report says 18.5% instead of 15.*	Valeur décimale tronquée et toxicité résiduelle.

8.3 Analyse des mécanismes d'erreur

Trois tendances ressortent : (i) la suppression locale de mots injurieux laisse parfois une structure mal formée ; (ii) les expressions idiomatiques nécessitent une réécriture de plusieurs tokens ; (iii) la préservation des faits dépend d'unités que le tokenizer et les heuristiques ne comprennent pas toujours. Les sorties incorrectes peuvent ne pas être détectées par `Needs_review`, car des règles numériques ne constituent pas un vérificateur grammatical.

Le notebook comprend **60 diagnostics initiaux** et **40 diagnostics supplémentaires**. Ils servent à examiner des cas délicats (négations, dates, noms, phrases neutres et longues), *pas* à estimer une précision statistiquement représentative. Une évaluation humaine indépendante reste nécessaire pour noter séparément style, fidélité et grammaticalité.

9. Discussion critique

9.1 Réponses aux questions expérimentales

Le modèle apprend-il la tâche ?
Oui, au sens où les variantes avec attention surpassent fortement la baseline en recouvrement et en fidélité moyenne, tout en réduisant la toxicité prédite. Les erreurs qualitatives prouvent toutefois que la tâche n'est pas résolue.

L'attention aide-t-elle ?
Les résultats expérimentaux montrent un avantage marqué de l'Attention par rapport au modèle sans attention, avec un budget d'apprentissage comparable pour les runs initiaux.

Le Pointer est-il toujours supérieur ?
Non. En greedy, il peut conserver davantage d'information mais aussi de toxicité. C'est l'association avec le *Safe Decoding* qui obtient le meilleur compromis dans le protocole exécuté.

Y a-t-il du surapprentissage ?
Les courbes montrent des écarts train–validation et un meilleur checkpoint Pointer précoce. Elles suggèrent de surveiller la généralisation, mais ne suffisent pas seules pour diagnostiquer définitivement un surapprentissage.

Quels cas sont difficiles ?
Idiomes, négations, dates, décimales, noms et phrases longues ; ils sollicitent simultanément la correction stylistique et la conservation du contenu.

9.2 Limites méthodologiques

Le projet reste limité par un corpus de taille modérée, un vocabulaire de 2 000 sous-mots, une seule graine d'entraînement, l'absence de notation humaine exhaustive et des métriques automatiques imparfaites. L'extension Attention ne peut pas être comparée de manière strictement causale aux autres modèles, car elle modifie le budget et le teacher forcing. Le choix de plusieurs stratégies sur validation réduit le risque de sélection sur le test, mais les itérations antérieures et les diagnostics consultés pendant le développement imposent une prudence supplémentaire dans l'interprétation des chiffres.

9.3 Améliorations futures

Les pistes prioritaires seraient : plusieurs graines et intervalles sur les runs, contrôle plus précis des entités et relations numériques, annotation humaine aveugle de la grammaire et du sens, jeux de tests de généralisation, et apprentissage de reformulations d'expressions complètes. L'ajout d'un *coverage mechanism* ou de contraintes factuelles mieux motivées pourrait être étudié. Ces pistes sont **proposées**, non présentées comme déjà implémentées ou validées.

10. Conclusion

Ce projet réalise une chaîne NLP complète : nettoyage d'un corpus parallèle, apprentissage d'un tokenizer, construction de modèles Encoder–Decoder depuis zéro, entraînement supervisé, génération autorégressive et comparaison quantitative et qualitative. L'attention améliore nettement la génération par rapport à la baseline, tandis que le Pointer associé à un reclassement prudent de candidats obtient les meilleurs scores de l'expérience courante.

Le système final atteint **BLEU = 66,59**, **chrF = 79,88** et **66,81 %** sur le proxy conjoint automatique, avec des limites explicites : ce n'est ni une vérification humaine ni la preuve de l'absence d'erreurs. Les échecs observés montrent que *préserver le sens, réduire l'offense et garder une phrase grammaticale* sont trois contraintes simultanées difficiles. L'apport académique principal réside dans la compréhension de ces mécanismes, l'implémentation reproductible et l'analyse critique de leurs compromis.

Traçabilité et reproductibilité

Source primaire des expériences : `ParaDetox_FINAL_REMIS_REDACTION.ipynb`, version exécutée de la famille V3. Données et tokenizer chargés ou entraînés par le code ; figures de ce rapport extraites **des sorties réellement enregistrées**, sans reconstruction des scores. Environnement observé : Google Colab, GPU Tesla T4, Python 3.13.16, PyTorch 2.11.0+cu130, graine 42. Les checkpoints ne sont pas intégrés au notebook ; une nouvelle exécution complète peut produire des résultats légèrement différents.

Références

[1] V. Logacheva, D. Dementieva, S. Ustyantsev, D. Moskovskiy, D. Dale, I. Krotova, N. Semenov, A. Panchenko, *ParaDetox : Detoxification with Parallel Data*, ACL, 2022. https://aclanthology.org/2022.acl-long.469/.

[2] I. Sutskever, O. Vinyals, Q. V. Le, *Sequence to Sequence Learning with Neural Networks*, NeurIPS, 2014. https://arxiv.org/abs/1409.3215.

[3] D. Bahdanau, K. Cho, Y. Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*, ICLR, 2015. https://arxiv.org/abs/1409.0473.

[4] A. See, P. J. Liu, C. D. Manning, *Get To The Point : Summarization with Pointer-Generator Networks*, ACL, 2017. https://aclanthology.org/P17-1099/.

[5] S. Bengio, O. Vinyals, N. Jaitly, N. Shazeer, *Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks*, NeurIPS, 2015. https://proceedings.neurips.cc/paper/2015/hash/e995f98d56967d946471af29d7bf99f1-Abstract.html.

[6] T. Kudo, J. Richardson, *SentencePiece : A simple and language independent subword tokenizer and detokenizer for Neural Text Processing*, EMNLP, 2018. https://aclanthology.org/D18-2012/.